
Chapter 3 | GEMM 控制器与卷积数据通路

- 本章目标：让脉动阵列真正计算卷积。
- 内容：SRAM、控制器 FSM、im2col/line buffer、激活/池化/FC。
- 验收：小规模 golden 逐元素一致。

Chapter 3 | 阵列输出的意义

- $C[k][col] = \text{sum_row } A[\text{row}][k] * W[\text{row}][col]$ 。
- 把 A 的行当作输入通道、列当作空间位置。
- W 的行是输入通道、列是输出通道。
- 所以一拍算出一个像素的 N 个输出通道。

Chapter 3 | SRAM 组织

- 输入 SRAM : $H \times W \times C_{in}$, 按通道-空间布局。
- 权重 SRAM : $C_{out} \times C_{in} \times 3 \times 3$, 按 im2col 后矩阵布局。
- 输出 SRAM : $H \times W \times C_{out}$ 。
- 规模估算 : $28 \times 28 \times 8 = 6272$ 字节, int8。

Chapter 3 | 控制器 FSM

- IDLE：等待 CMD。
- LOAD_W：从权重 SRAM 装载阵列。
- RUN：逐像素产生 a_data 并收集 psum_out。
- DRAIN：等待流水排空。
- DONE：置位状态寄存器。

Chapter 3 | im2col

- 把 3x3 卷积窗口展开成行向量。
- 每个输出像素对应一个 $K = C_{in} * 3 * 3$ 维向量。
- 一行一行喂给阵列，等价于矩阵乘。
- 代价是数据重复，但硬件实现简单。

Chapter 3 | Line Buffer

- 逐行扫描图片时，只需要保留最近 3 行。
- line buffer 可以避免整张图重复搬运。
- 适合流式卷积，减少 SRAM 带宽。
- 两种方案都可以，本章不强制。

Chapter 3 | ReLU 与量化

- int32 累加结果需要反量化并重新量化到 int8。
- ReLU 在量化域里就是 $\max(x, 0)$ 。
- LeakyReLU 需要查表或移位近似。
- 本章先用普通 ReLU。

Chapter 3 | MaxPool

- 2x2 窗口取最大值, stride=2 时宽高减半。
- 可以跟在卷积输出后做流式处理。
- 实现简单：比较器 + 计数器。
- Pooling 不做乘法, 不占用阵列。

Chapter 3 | 全连接 = GEMM

- FC 层就是矩阵乘：输入向量 $\times$ 权重矩阵。
- 复用同一个控制器，只是没有滑动窗口。
- MNIST 小模型最后的 $784 \rightarrow 64 \rightarrow 10$ 都是 FC。
- 阵列位宽不足时按 8 通道 tiling。

Chapter 3 | Golden 验证

- 用 Python 写 int8 参考实现。
- 每层输出导出为 hex 文件。
- RTL testbench 逐层比对。
- 先单层验证，再做层间串联。

Chapter 3 | 本章任务

- 实现卷积控制器与数据通路。
- 实现 ReLU 与 MaxPool。
- 实现 FC 层。
- 与 Python golden 逐元素一致。

Chapter 3 | 总结

- 控制器决定“什么时候把什么数据送到阵列”。
- im2col 与 line buffer 是两种空间换带宽的方案。
- 激活/池化跟在累加之后，仍是数据流的一部分。
- 下一章解决“权重从哪来”：模型训练与量化导出。